Coloured Graphs as Directed Containers

with an application to INSTINCT

Robin Langer

July 2026

Abstract

We show that every coloured directed graph carries a canonical directed container structure, whose positions are typed directed paths and whose comonad is the path-context comonad. Node and edge colours ride as passengers on the path algebra, stratifying positions into typed sub-containers without altering the five directed container laws. We apply this to INSTINCT, a knowledge-graph pipeline for research papers, and identify its three main operations—concept summarisation, trust propagation, and structural hole detection—as coKleisli morphisms, with the coKleisli composition law giving compositionality for free.

1 Coloured graphs as directed containers

1.1 Coloured directed graphs

Definition 1.1 (Coloured directed graph)**.** A *coloured directed graph* is a tuple $G = (V, E, \tau, \lambda, \mathrm{src}, \mathrm{tgt})$ where V is a set of *vertices*, E is a set of *edges*, $\tau : V \to C_V$ assigns each vertex a *node colour*, $\lambda : E \to C_E$ assigns each edge an *edge colour*, and $\mathrm{src}, \mathrm{tgt} : E \to V$ are the source and target maps.

The colour sets C_V and C_E are arbitrary; in applications they are finite enumerations of types and relation labels.

1.2 The container

Definition 1.2 (Container (Abbott–Altenkirk–Ghani))**.** A *container* is a pair (S, P) where S is a set of *shapes* and $P : S \to \mathbf{Set}$ is a family of *positions*. Its *extension* is $C(X) = \sum_{s:S}(P(s) \to X)$.

Definition 1.3 (Paths)**.** Let $v \in V$. A *path from* v is a finite sequence of edges $p = e_1 e_2 \cdots e_n$ with $\mathrm{src}(e_1) = v$ and $\mathrm{tgt}(e_i) = \mathrm{src}(e_{i+1})$. The *empty path* $() \in P(v)$ is included for every v. Write $\mathrm{tgt}(p) = \mathrm{tgt}(e_n)$ and $\mathrm{tgt}(()) = v$. Denote the set of all paths from v by $P(v)$.

The pair (V, P) is a container with $P : V \to \mathbf{Set}$.

Remark 1.4 (Why paths, not nodes). A position is not merely a reachable node but the *route* taken to reach it. Consider a graph with three nodes and edges $A \xrightarrow{\texttt{proves}} B \xrightarrow{\texttt{uses}} C$ and $A \xrightarrow{\texttt{related_to}} C$. The positions in $P(A)$ are:

$$(), \quad [A \to B], \quad [A \to C], \quad [A \to B, B \to C].$$

The last two positions both end at C, but they are distinct elements of $P(A)$ because they took different routes. In a *tree*, there is exactly one path between any two nodes, so position and destination coincide; this is why tree-shaped directed containers (such as proof registries) can treat nodes as positions. In a general graph, multiple paths may connect the same pair of nodes, so the position must record the full route. Consequently, $W(X)(A) = P(A) \to X$ can assign different X-values to C depending on whether it was reached via `related_to` or via $\texttt{proves} \cdot \texttt{uses}$ — a distinction that is invisible if positions are merely nodes.

1.3 The directed container structure

Definition 1.5 (Directed container (Ahman–Chapman–Uustalu))**.** A *directed container* is a container (S, P) equipped with a *root* $\mathsf{root}(s) \in P(s)$, a *sub-shape* $s \downarrow p \in S$ for $p \in P(s)$, and a *shift* $p \oplus q \in P(s)$ for $q \in P(s \downarrow p)$, subject to five laws:

$$s \downarrow \mathsf{root}(s) = s \tag{D1}$$
$$\mathsf{root}(s) \oplus q = q \tag{D2}$$
$$p \oplus \mathsf{root}(s \downarrow p) = p \tag{D3}$$
$$s \downarrow (p \oplus q) = (s \downarrow p) \downarrow q \tag{D4}$$
$$(p \oplus q) \oplus r = p \oplus (q \oplus r) \tag{D5}$$

Theorem 1.6. *Every coloured directed graph G determines a directed container (V, P) with*

$$\mathsf{root}(v) = (), \qquad v \downarrow p = \mathrm{tgt}(p), \qquad p \oplus q = pq \text{ (concatenation).}$$

Proof. D1: $v \downarrow () = v$. D2: $() \cdot q = q$. D3: $p \cdot () = p$. D4: $\mathrm{tgt}(pq) = \mathrm{tgt}(q) = \mathrm{tgt}(p) \downarrow q$. D5: concatenation is associative. All hold by definition. □

1.4 Indexed containers and node colours

Node colours do more than label shapes: they turn (V, P) into an *indexed* container, where each position carries the colour of its endpoint as a type.

Definition 1.7 (Indexed container over I)**.** A *container indexed over I* is a triple (S, P, r) where $S : I \to \mathbf{Set}$ assigns a set of shapes to each index, $P : \prod_{i:I} S(i) \to \mathbf{Set}$ assigns positions to each indexed shape, and $r : \prod_{i:I} \prod_{s:S(i)} P(i, s) \to I$ is the *response map*, assigning to each position the index of its sub-shape. Its extension is the endofunctor on $\mathbf{Set}^I$:

$$C(X)(i) = \sum_{s:S(i)} \prod_{p:P(i,s)} X\big(r(i, s, p)\big).$$

An *indexed directed container* additionally carries root, $\downarrow$, $\oplus$ satisfying D1–D5, with the constraint that the root position stays at its own index: $r(i, s, \mathsf{root}(i, s)) = i$.

Proposition 1.8 (C_V-indexed directed container)**.** *The coloured graph G determines a C_V-indexed directed container with:*

$$S(c) = \{v \in V : \tau(v) = c\},$$
$$P(c, v) = \textit{paths from } v \textit{ in } G,$$
$$r(c, v, p) = \tau(\mathrm{tgt}(p)).$$

The operations root, $\downarrow$, $\oplus$ are as in Theorem 1.6.

Proof. The root condition holds: $r(c, v, ()) = \tau(\mathrm{tgt}()) = \tau(v) = c$. Shift is index-compatible: if $r(c, v, p) = c'$ and $r(c', v \downarrow p, q) = c''$ then $r(c, v, p \oplus q) = \tau(\mathrm{tgt}(pq)) = c''$. D1–D5 are unchanged from Theorem 1.6. □

The extension of this indexed container is:

$$C(X)(c) = \sum_{v:S(c)} \prod_{p:P(c,v)} X\big(\tau(v \downarrow p)\big).$$

A filling $\langle v, f \rangle$ assigns to each path p from v a value in $X(\tau(v \downarrow p))$ — a value *typed by the colour of the endpoint*. This is a strictly richer structure than an unindexed filling $P(v) \to X$.

1.5 Edge colours as a grading

Edge colours play a different role: they do not index the container but *grade* the position sets.

Definition 1.9 (Colour word)**.** For a path $p = e_1 \cdots e_n$, its *colour word* is $w(p) = \lambda(e_1) \cdots \lambda(e_n) \in C_E^*$. The colour word of the empty path is ε.

The map w satisfies $w(p \oplus q) = w(p) \cdot w(q)$, so $P(c, v) = \bigsqcup_{w \in C_E^*} P_w(c, v)$ where $P_w(c, v) = \{p \in P(c, v) : w(p) = w\}$.

Proposition 1.10 (Sub-containers from colour classes)**.** *For any $C' \subseteq C_E$, the family $P_{C'}(c, v) = \{p \in P(c, v) : w(p) \in (C')^*\}$ is a C_V-indexed directed sub-container, with the same* root*, $\downarrow$, $\oplus$, r.*

Proof. $() \in P_{C'}(c, v)$ since $\varepsilon \in (C')^*$; the root condition is inherited. Shift and the response map restrict since $(C')^*$ is closed under concatenation. D1–D5 hold by restriction. □

Remark 1.11 (Node colours vs edge colours). The two colour types play structurally distinct roles. Node colours *index* the container: they partition the shape space and type each position by its endpoint colour, turning the comonad from one on **Set** to one on $\mathbf{Set}^{C_V}$. Edge colours *grade* the position sets: they stratify $P(c, v)$ into sub-containers $P_{C'}$ without changing the index structure. In directed containers for trust boundaries [4], trust labels ride as true passengers on rose trees — they neither index nor grade, but are carried unchanged by every operation. Here neither colour type is a pure passenger: node colours index, edge colours grade.

1.6 The comonad and coKleisli morphisms

By Ahman–Chapman–Uustalu [2], directed containers are exactly containers whose extension functor carries a comonad.

Proposition 1.12 (Path-context comonad)**.** *The comonad on $W(X)(v) = P(v) \to X$ has operations:*

$$\varepsilon\langle v, f\rangle = f() \qquad \textit{(value at current node)}$$
$$\delta\langle v, f\rangle = \langle v,\ \lambda p.\ \langle v \downarrow p,\ \lambda q.\ f(p \oplus q)\rangle\rangle \qquad \textit{(at each reachable node, see all onward paths)}$$

The three comonad laws follow from D1–D5.

Definition 1.13 (CoKleisli morphism)**.** A *coKleisli morphism* $f : W(X) \to Y$ is a function that at each node v inspects the full typed-path context — the values $h(p)$ for all $p \in P(v)$, their colour words $w(p)$, and endpoint colours $\tau(v \downarrow p)$ — before producing a value in Y. The coKleisli extension of f is

$$\mathrm{extend}(f)\langle v, h\rangle = \langle v,\ \lambda p.\ f\langle v \downarrow p,\ \lambda q.\ h(p \oplus q)\rangle\rangle,$$

and coKleisli composition is $g \bullet f = g \circ \mathrm{extend}(f)$.

2 Application to INSTINCT

INSTINCT is a six-stage pipeline that ingests research papers and produces a coloured directed graph of mathematical concepts and their relationships.

2.1 The INSTINCT graph

The colour sets are:

$C_V = \{\texttt{theorem}, \texttt{object}, \texttt{conjecture}, \texttt{technique}, \texttt{identity}, \texttt{formula}, \texttt{person}, \texttt{definition}\}$,
$C_E = \{\texttt{proves}, \texttt{generalizes}, \texttt{uses}, \texttt{implies}, \texttt{is_instance_of}, \texttt{conjectured_by}, \texttt{related_to}, \texttt{equivalent_to}, \texttt{specializes_to}, \texttt{defined_in}\}$.

Nodes are mathematical concepts extracted from papers by an LLM; edges record how concepts relate. By Theorem 1.6, this is a directed container (V, P) with shapes = concepts, positions = typed paths, and the path comonad.

2.2 Colour words as query types

Single-colour-class sub-containers correspond to natural query families:

Sub-container	Colour class C'	Meaning
$(V, P_{\{\texttt{proves}\}})$	$\{\texttt{proves}\}$	concepts reachable by chains of proofs
$(V, P_{\{\texttt{generalizes}\}})$	$\{\texttt{generalizes}\}$	generalisation chains
$(V, P_{\{\texttt{proves},\texttt{uses}\}})$	$\{\texttt{proves},\texttt{uses}\}$	proofs and their dependencies
$(V, P_{\{\texttt{implies},\texttt{related_to}\}})$	$\{\texttt{implies},\texttt{related_to}\}$	soft semantic neighbourhood

Each row is a directed sub-container. Restricting a coKleisli morphism to $P_{C'}$ gives a typed query that traverses only semantically appropriate edges.

2.3 Three coKleisli morphisms

Concept summarisation. INSTINCT currently generates per-concept summaries by querying a vector database (ChromaDB) for embedding-similar passages. The directed container formulation gives a complementary, structurally-aware approach: a coKleisli morphism $f : W(\mathsf{desc}) \to \mathsf{summary}$ that aggregates descriptions at nodes reachable by typed paths of bounded length:

$$f\langle v, d\rangle = \mathsf{LLM}\big(v,\ \big\{(w(p),\ d(p)) : p \in P_{\{\texttt{proves},\texttt{uses},\texttt{generalizes}\}}(v),\ |p| \le 2\big\}\big)\,.$$

The vector database retrieves concepts that *use the same terminology*; the directed container retrieves concepts that are *structurally related*. The two are complementary: the former captures terminological similarity, the latter captures dependency structure.

Trust propagation. The edge colours carry an implicit reliability ordering: `proves` > `implies` > `related_to`.
Fix a base confidence $\kappa : V \to [0, 1]$ and a decay factor $\alpha \in (0, 1)$. The coKleisli morphism $f : W(\kappa) \to [0, 1]$,

$$f\langle v, \kappa\rangle = \min_{\substack{p \in P_{\{\texttt{proves},\texttt{implies}\}}(v) \\ p \neq ()}} \kappa(v \downarrow p) \cdot \alpha^{|p|},$$

propagates trust laxly along proof and implication paths. The comonad laws ensure soundness: trust at v cannot exceed the transported trust of any supporting concept. This mirrors Lemma 6.1 of [4], where trust transports laxly along directed container morphisms.

Structural holes. A *structural hole* at v is a concept $w = v \downarrow p$ reachable by a path of length ≥ 2 but not directly connected to v:

$$\mathsf{Holes}(v) = \big\{ v \downarrow p : p \in P(v),\ |p| \geq 2,\ v \downarrow p \notin \{\mathrm{tgt}(e) : \mathrm{src}(e) = v\} \big\}.$$

This is a coKleisli morphism $W(V) \to \mathcal{P}(V)$ that inspects the two-step neighbourhood and subtracts the one-step neighbourhood. The INSTINCT pipeline already contains a structural hole detector (`kg/structural_holes.py`); the directed container formulation makes its semantics precise and its composition with other morphisms principled.

2.4 Composition

The coKleisli composition law $g \bullet f = g \circ \mathrm{extend}(f)$ allows the three morphisms above to be chained. The composite

$$\text{trust propagation} \bullet \text{structural hole detection} \bullet \text{concept summarisation}$$

is a single coKleisli morphism $W(\mathsf{desc}) \to \mathsf{annotated_summary}$ that summarises a concept, identifies gaps in its structural neighbourhood, and weights conclusions by the propagated trust of supporting concepts. Correctness of this composition — that the pipeline does not lose information or violate trust monotonicity — is a consequence of the three comonad laws, which themselves follow from D1–D5 on the underlying path structure.

References

[1] M. Abbott, T. Altenkirk, and N. Ghani. Containers: Constructing strictly positive types. *Theoretical Computer Science*, 342:3–27, 2005.

[2] D. Ahman, J. Chapman, and T. Uustalu. When is a container a comonad? In *FoSSaCS*, LNCS 7213, 2012.

[3] D. Ahman and T. Uustalu. Directed containers as categories. *EPTCS*, 207:89–98, 2016.

[4] R. Langer. Directed containers in trust boundaries: How proof registries are comonads. 2026.